

COIN CHANGE PROBLEM

```

def coin_change(coins, amount):
    """
    Coin Change Problem using Dynamic Programming
    """

    # dp[i] = minimum number of coins needed
    # to make amount i
    dp = [float('inf')] * (amount + 1)

    # Base case
    dp[0] = 0

    # Calculate minimum coins
    for i in range(1, amount + 1):
        for coin in coins:
            if coin <= i:
                dp[i] = min(dp[i], dp[i - coin] + 1)

    # If amount cannot be formed
    if dp[amount] == float('inf'):
        return -1

    return dp[amount]

# -----
# MAIN PROGRAM
# -----

coins = [1, 2, 5]
amount = 11

result = coin_change(coins, amount)

print("Coins:", coins)
print("Amount:", amount)
print("Minimum number of coins:", result)

# -----
# COMPLEXITY
# -----

n = amount
m = len(coins)

print("\n----- COMPLEXITY ANALYSTS -----")

```

```
print("Best Case Time Complexity    : O(n * m)")
print("Average Case Time Complexity : O(n * m)")
print("Worst Case Time Complexity   : O(n * m)")

print("Best Case Space Complexity    : O(n)")
print("Average Case Space Complexity : O(n)")
print("Worst Case Space Complexity   : O(n)")

print("\nFor this input:")
print("n =", n)
print("m =", m)
print("Time Complexity : O(", n, "*", m, ")")
print("Space Complexity: O(", n, ")")
```

Coins: [1, 2, 5]
Amount: 11
Minimum number of coins: 3

----- COMPLEXITY ANALYSIS -----
Best Case Time Complexity : O(n * m)
Average Case Time Complexity : O(n * m)
Worst Case Time Complexity : O(n * m)
Best Case Space Complexity : O(n)
Average Case Space Complexity : O(n)
Worst Case Space Complexity : O(n)

For this input:
n = 11
m = 3
Time Complexity : O(11 * 3)
Space Complexity: O(11)

